IPL Fielding Performance Analysis

IPL Fielding Performance Dataset

Project Overview

Analysis of IPL fielding performance using Python

Tools: Pandas, NumPy, Matplotlib and Seaborn.

Focus: Performance Score (PS), fielding actions

and player-wise comparison.

IPL Fielding Performance Analysis Report

- Python
- Pandas
- Matplotlib
- Seaborn

Data Loading

Code

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

Output

No displayed output was produced by this cell.

Explanation

Imports the libraries required for the IPL fielding analysis.

Code

```
from google.colab import files
import pandas as pd
import io

uploaded = files.upload()

file_name = next(iter(uploaded))
df = pd.read_excel(io.BytesIO(uploaded[file_name]))

print("Uploaded file:", file_name)
print("Dataset loaded successfully.")
```

Output

```
<IPython.core.display.HTML object>



Saving IPL sample data.xlsx - Google Sheets.pdf to IPL sample data.xlsx - Google Sheets.pdf
```

Explanation

Uploads and loads the project dataset into a Pandas dataframe.

Initial Data View

Code

```
df.shape
```

Output

```
(7, 11)
```

Explanation

Inspects the dataframe structure, rows, columns and basic information.

Code

```
df.columns
```

Output

```
Index(['Player Name', 'CP', 'GT', 'C', 'DC', 'ST', 'RO', 'MRO', 'DH', 'RS',
       'PS'],
      dtype='object')
```

Explanation

Inspects the dataframe structure, rows, columns and basic information.

Code

```
df.info()
```

Output

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 7 entries, 0 to 6
Data columns (total 11 columns):
 #   Column       Non-Null Count  Dtype 
---  ------       --------------  ----- 
 0   Player Name  7 non-null      object
 1   CP           7 non-null      int64 
 2   GT           7 non-null      int64 
 3   C            7 non-null      int64 
 4   DC           7 non-null      int64 
 5   ST           7 non-null      int64 
 6   RO           7 non-null      int64 
 7   MRO          7 non-null      int64 
 8   DH           7 non-null      int64 
 9   RS           7 non-null      int64 
 10  PS           7 non-null      int64 
dtypes: int64(10), object(1)
memory usage: 748.0+ bytes
```

Explanation

Inspects the dataframe structure, rows, columns and basic information.

Data Information

Code

```
df.describe()
```

Output

```
              CP        GT         C        DC        ST        RO       MRO  \
count  7.000000  7.000000  7.000000  7.000000  7.000000  7.000000  7.000000
mean   2.285714  1.428571  0.857143  0.285714  0.142857  0.285714  0.142857
std    1.112697  0.975900  0.690066  0.487950  0.377964  0.487950  0.377964
min    1.000000  0.000000  0.000000  0.000000  0.000000  0.000000  0.000000
25%    1.500000  1.000000  0.500000  0.000000  0.000000  0.000000  0.000000
50%    2.000000  1.000000  1.000000  0.000000  0.000000  0.000000  0.000000
75%    3.000000  2.000000  1.000000  0.500000  0.000000  0.500000  0.000000
max    4.000000  3.000000  2.000000  1.000000  1.000000  1.000000  1.000000

              DH        RS         PS
count  7.000000  7.000000   7.000000
mean   0.428571  1.000000   8.285714
std    0.534522  2.160247   3.251373
min    0.000000 -2.000000   2.000000
25%    0.000000 -0.500000   7.500000
50%    0.000000  1.000000   9.000000
75%    1.000000  2.500000  10.500000
max    1.000000  4.000000  11.000000
```

Explanation

Provides descriptive statistics and data-quality information for the dataset.

Code

```
df.isnull().sum()
```

Output

```
Player Name    0
CP             0
GT             0
C              0
DC             0
ST             0
RO             0
MRO            0
DH             0
RS             0
PS             0
dtype: int64
```

Explanation

Performs the corresponding analysis step from the Advance Project notebook.

Code

```
df.duplicated().sum()
```

Output

```
np.int64(0)
```

Explanation

Performs the corresponding analysis step from the Advance Project notebook.

Dataset Columns

Code

```
df.head(10)
```

Output

```
     Player Name  CP  GT  C  DC  ST  RO  MRO  DH  RS  PS
0  Rilee russouw   2   1  1   0   0   0    0   1   2  10
1      Phil Salt   1   2  0   1   0   1    0   0  -1   2
2     Yash Dhull   3   1  2   0   0   0    1   0   3  11
3     Axer Patel   2   3  1   0   1   0    0   0   0  11
4    Lalit yadav   1   2  1   0   0   0    0   1  -2   6
5      Aman Khan   4   1  0   0   0   1    0   0   1   9
6  Kuldeep yadav   3   0  1   1   0   0    0   1   4   9
```

Explanation

Inspects the dataframe structure, rows, columns and basic information.

Code

```
df.columns
```

Output

```
Index(['Player Name', 'CP', 'GT', 'C', 'DC', 'ST', 'RO', 'MRO', 'DH', 'RS',
       'PS'],
      dtype='object')
```

Explanation

Inspects the dataframe structure, rows, columns and basic information.

Code

```
df.head()
```

Output

```
     Player Name  CP  GT  C  DC  ST  RO  MRO  DH  RS  PS
0  Rilee russouw   2   1  1   0   0   0    0   1   2  10
1      Phil Salt   1   2  0   1   0   1    0   0  -1   2
2     Yash Dhull   3   1  2   0   0   0    1   0   3  11
3     Axer Patel   2   3  1   0   1   0    0   0   0  11
4    Lalit yadav   1   2  1   0   0   0    0   1  -2   6
```

Explanation

Inspects the dataframe structure, rows, columns and basic information.

Code

```
df.info()
```

Output

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 7 entries, 0 to 6
Data columns (total 11 columns):
 #   Column       Non-Null Count  Dtype 
---  ------       --------------  ----- 
 0   Player Name  7 non-null      object
 1   CP           7 non-null      int64 
 2   GT           7 non-null      int64 
 3   C            7 non-null      int64 
 4   DC           7 non-null      int64 
 5   ST           7 non-null      int64 
 6   RO           7 non-null      int64 
 7   MRO          7 non-null      int64 
 8   DH           7 non-null      int64 
 9   RS           7 non-null      int64 
 10  PS           7 non-null      int64 
dtypes: int64(10), object(1)
memory usage: 748.0+ bytes
```

Explanation

Inspects the dataframe structure, rows, columns and basic information.

Descriptive Analysis

Code

```
df.describe()
```

Output

```
              CP        GT         C        DC        ST        RO       MRO  \
count  7.000000  7.000000  7.000000  7.000000  7.000000  7.000000  7.000000
mean   2.285714  1.428571  0.857143  0.285714  0.142857  0.285714  0.142857
std    1.112697  0.975900  0.690066  0.487950  0.377964  0.487950  0.377964
min    1.000000  0.000000  0.000000  0.000000  0.000000  0.000000  0.000000
25%    1.500000  1.000000  0.500000  0.000000  0.000000  0.000000  0.000000
50%    2.000000  1.000000  1.000000  0.000000  0.000000  0.000000  0.000000
75%    3.000000  2.000000  1.000000  0.500000  0.000000  0.500000  0.000000
max    4.000000  3.000000  2.000000  1.000000  1.000000  1.000000  1.000000

              DH        RS         PS
count  7.000000  7.000000   7.000000
mean   0.428571  1.000000   8.285714
std    0.534522  2.160247   3.251373
min    0.000000 -2.000000   2.000000
25%    0.000000 -0.500000   7.500000
50%    0.000000  1.000000   9.000000
75%    1.000000  2.500000  10.500000
max    1.000000  4.000000  11.000000
```

Explanation

Provides descriptive statistics and data-quality information for the dataset.

Code

```
df.isnull().sum()
```

Output

```
Player Name    0
CP             0
GT             0
C              0
DC             0
ST             0
RO             0
MRO            0
DH             0
RS             0
PS             0
dtype: int64
```

Explanation

Provides descriptive statistics and data-quality information for the dataset.

Code

```
df.duplicated().sum()
```

Output

```
np.int64(0)
```

Explanation

Provides descriptive statistics and data-quality information for the dataset.

Performance Score

Code

```
# Calculate Performance Score according to the PDF formula
# PS = (CP×1) + (GT×1) + (C×3) − (DC×3) + (ST×3) + (RO×3) − (MRO×2) + (DH×2) + RS
df["PS"] = (
    (df["CP"] * 1)
    + (df["GT"] * 1)
    + (df["C"] * 3)
    - (df["DC"] * 3)
    + (df["ST"] * 3)
    + (df["RO"] * 3)
    - (df["MRO"] * 2)
    + (df["DH"] * 2)
    + df["RS"]
)

df[["Player Name", "PS"]]
```

Output

```
     Player Name  PS
0  Rilee russouw  10
1      Phil Salt   2
2     Yash Dhull  11
3     Axer Patel  11
4    Lalit yadav   6
5      Aman Khan   9
6  Kuldeep yadav   9
```

Explanation

Calculates the Performance Score using the project's defined weighted formula.

Best Player

Code

```
df.loc[df["PS"].idxmax()]
```

Output

```
Player Name    Yash Dhull
CP                      3
GT                      1
C                       2
DC                      0
ST                      0
RO                      0
MRO                     1
DH                      0
RS                      3
PS                     11
Name: 2, dtype: object
```

Explanation

Identifies and compares the highest Performance Score in the dataset.

Code

```
df.sort_values(by="PS", ascending=False)
```

Output

	Player Name	CP	GT	C	DC	ST	RO	MRO	DH	RS	PS
2	Yash Dhull	3	1	2	0	0	0	1	0	3	11
3	Axer Patel	2	3	1	0	1	0	0	0	0	11
0	Rilee russouw	2	1	1	0	0	0	0	1	2	10
6	Kuldeep yadav	3	0	1	1	0	0	0	1	4	9
5	Aman Khan	4	1	0	0	0	1	0	0	1	9
4	Lalit yadav	1	2	1	0	0	0	0	1	-2	6
1	Phil Salt	1	2	0	1	0	1	0	0	-1	2

Explanation

Identifies and compares the highest Performance Score in the dataset.

Code

```
best_player = df.loc[df["PS"].idxmax(), "Player Name"]

print("Best Performing Player:", best_player)
```

Output

```
Best Performing Player: Yash Dhull
```

Explanation

Identifies and compares the highest Performance Score in the dataset.

Code

```
highest_score = df["PS"].max()

print("Highest Performance Score:", highest_score)
```

Output

```
Highest Performance Score: 11
```

Explanation

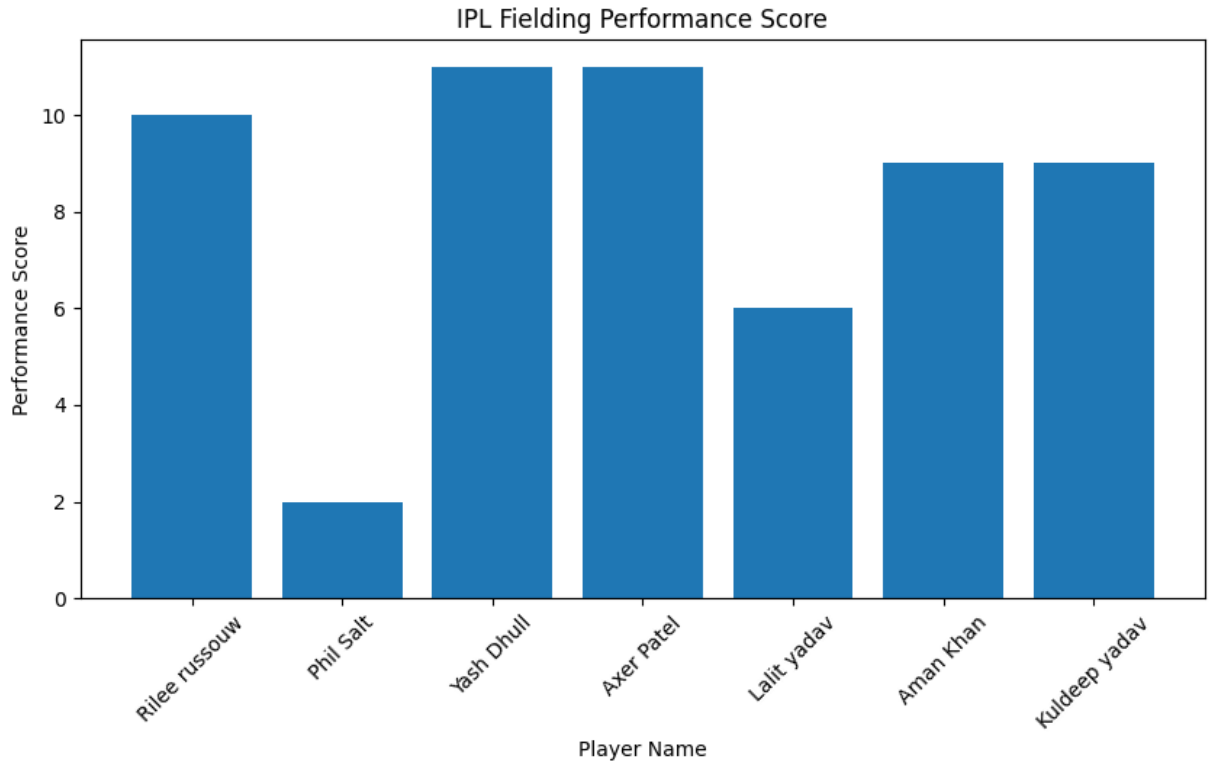
Identifies and compares the highest Performance Score in the dataset.

Performance Visualizations

Code

```
import matplotlib.pyplot as plt

plt.figure(figsize=(10,5))

plt.bar(df["Player Name"], df["PS"])

plt.xlabel("Player Name")
plt.ylabel("Performance Score")
plt.title("IPL Fielding Performance Score")

plt.xticks(rotation=45)

plt.show()
```

Output



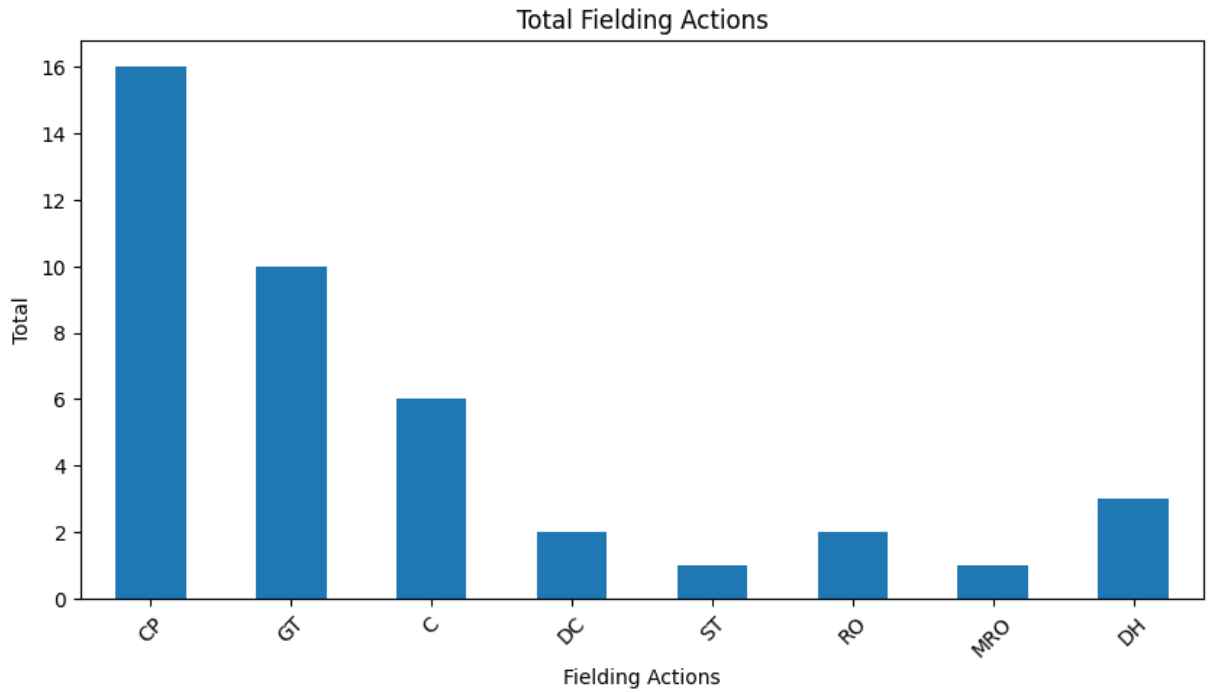
Explanation

Visualizes the selected fielding-performance measure for easier comparison.

Code

```
actions = ["CP", "GT", "C", "DC", "ST", "RO", "MRO", "DH"]

df[actions].sum().plot(
    kind="bar",
    figsize=(10,5)
)

plt.xlabel("Fielding Actions")
plt.ylabel("Total")
plt.title("Total Fielding Actions")

plt.xticks(rotation=45)

plt.show()
```

Output



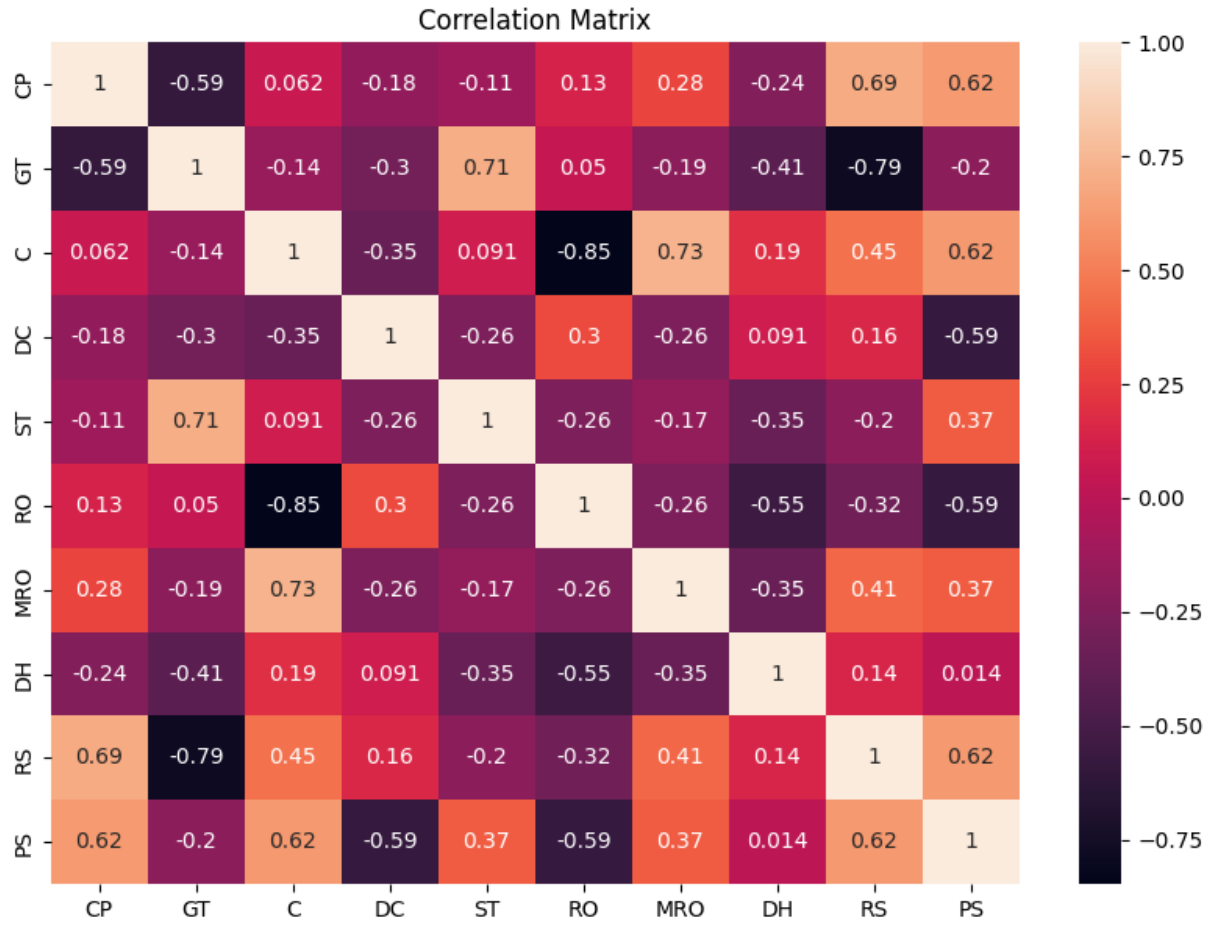
Explanation

Visualizes the selected fielding-performance measure for easier comparison.

Code

```
import seaborn as sns

plt.figure(figsize=(10,7))

sns.heatmap(
    df[["CP", "GT", "C", "DC", "ST", "RO", "MRO", "DH", "RS", "PS"]].corr(),
    annot=True
)

plt.title("Correlation Matrix")

plt.show()
```

Output



Explanation

Visualizes the selected fielding-performance measure for easier comparison.

Additional Visualizations

Code

```
df.sort_values(by="PS", ascending=False)
```

Output

	Player Name	CP	GT	C	DC	ST	RO	MRO	DH	RS	PS
2	Yash Dhull	3	1	2	0	0	0	1	0	3	11
3	Axer Patel	2	3	1	0	1	0	0	0	0	11
0	Rilee russouw	2	1	1	0	0	0	0	1	2	10
6	Kuldeep yadav	3	0	1	1	0	0	0	1	4	9
5	Aman Khan	4	1	0	0	0	1	0	0	1	9
4	Lalit yadav	1	2	1	0	0	0	0	1	-2	6
1	Phil Salt	1	2	0	1	0	1	0	0	-1	2

Explanation

Visualizes the selected fielding-performance measure for easier comparison.

Code

```
print("Average Performance Score:", df["PS"].mean())
print("Maximum Performance Score:", df["PS"].max())
print("Minimum Performance Score:", df["PS"].min())
print("Median Performance Score:", df["PS"].median())
```

Output

```
Average Performance Score: 8.285714285714286
Maximum Performance Score: 11
Minimum Performance Score: 2
Median Performance Score: 9.0
```

Explanation

Visualizes the selected fielding-performance measure for easier comparison.

Code

```
best_player = df.loc[df["PS"].idxmax()]

print("Best Performing Player:", best_player["Player Name"])
print("Performance Score:", best_player["PS"])
```

Output

```
Best Performing Player: Yash Dhull
Performance Score: 11
```

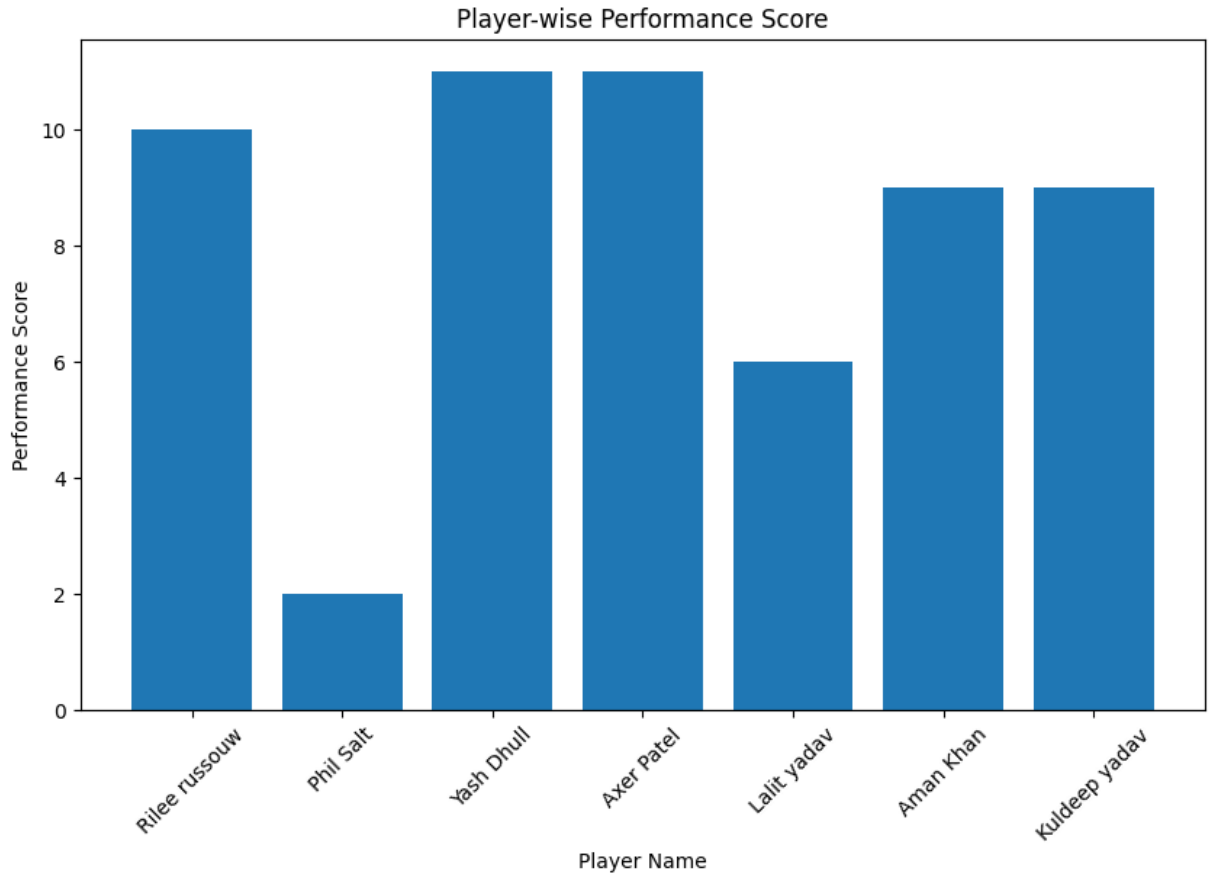
Explanation

Visualizes the selected fielding-performance measure for easier comparison.

Code

```
plt.figure(figsize=(10, 6))

plt.bar(df["Player Name"], df["PS"])

plt.title("Player-wise Performance Score")
plt.xlabel("Player Name")
plt.ylabel("Performance Score")

plt.xticks(rotation=45)

plt.show()
```

Output



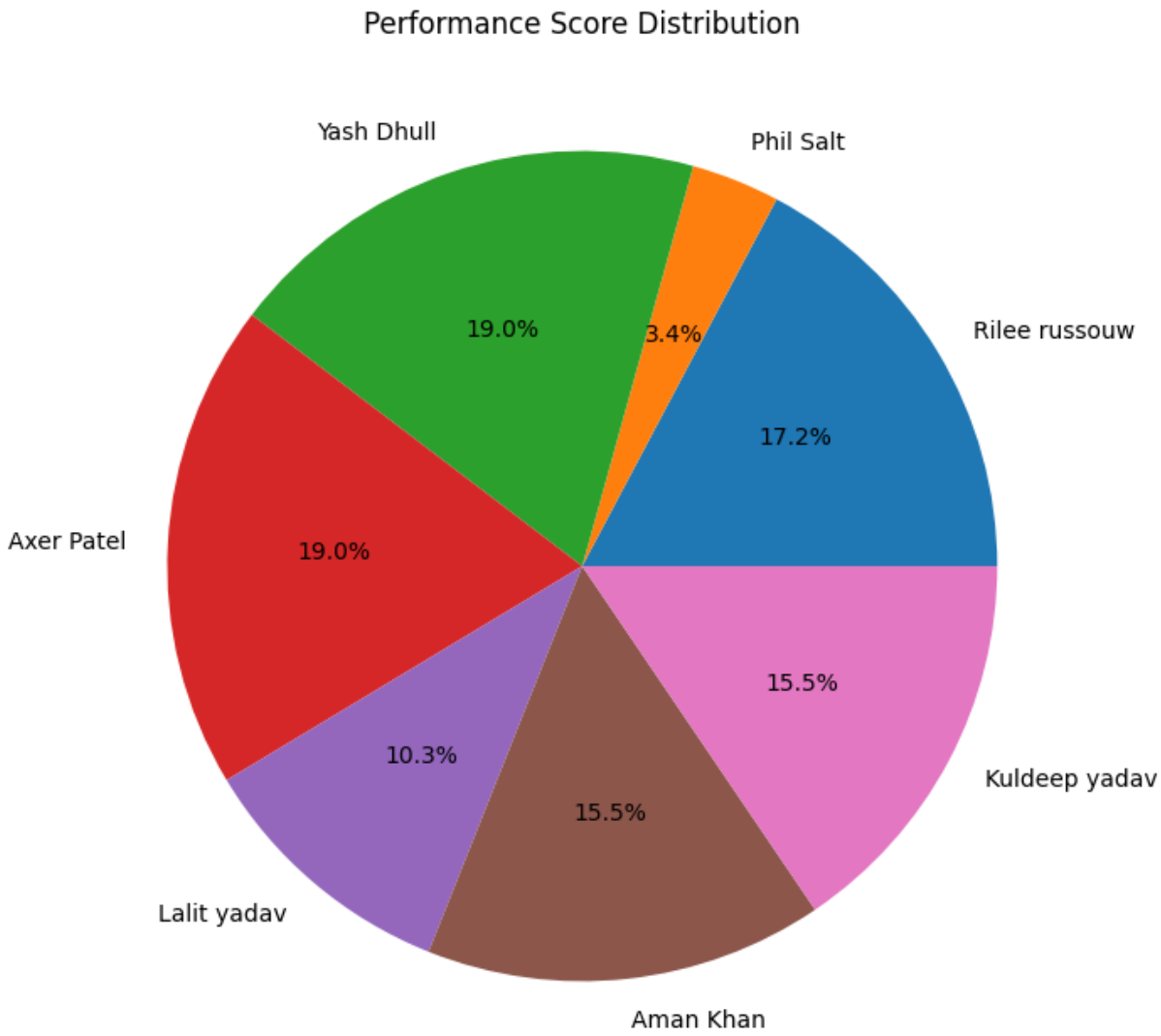
Explanation

Visualizes the selected fielding-performance measure for easier comparison.

Code

```
plt.figure(figsize=(8, 8))

plt.pie(
    df["PS"],
    labels=df["Player Name"],
    autopct="%1.1f%%"
)

plt.title("Performance Score Distribution")

plt.show()
```

Output



Explanation

Visualizes the selected fielding-performance measure for easier comparison.

Fielding Analysis

Code

```
actions = ["CP", "GT", "C", "DC", "ST", "RO", "MRO", "DH"]

action_totals = df[actions].sum()

action_totals
```

Output

```
CP     16
GT     10
C       6
DC      2
ST      1
RO      2
MRO     1
DH      3
dtype: int64
```

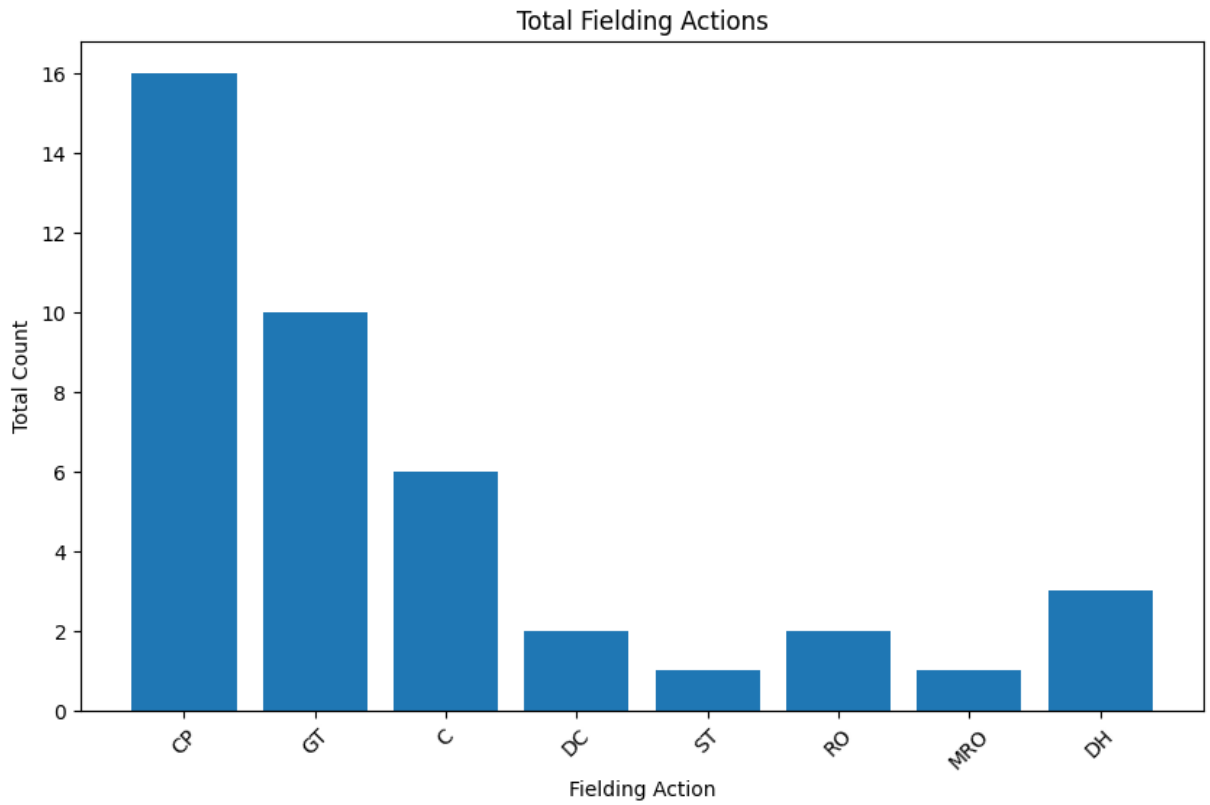
Explanation

Visualizes the selected fielding-performance measure for easier comparison.

Code

```
plt.figure(figsize=(10, 6))

plt.bar(action_totals.index, action_totals.values)

plt.title("Total Fielding Actions")
plt.xlabel("Fielding Action")
plt.ylabel("Total Count")

plt.xticks(rotation=45)

plt.show()
```

Output



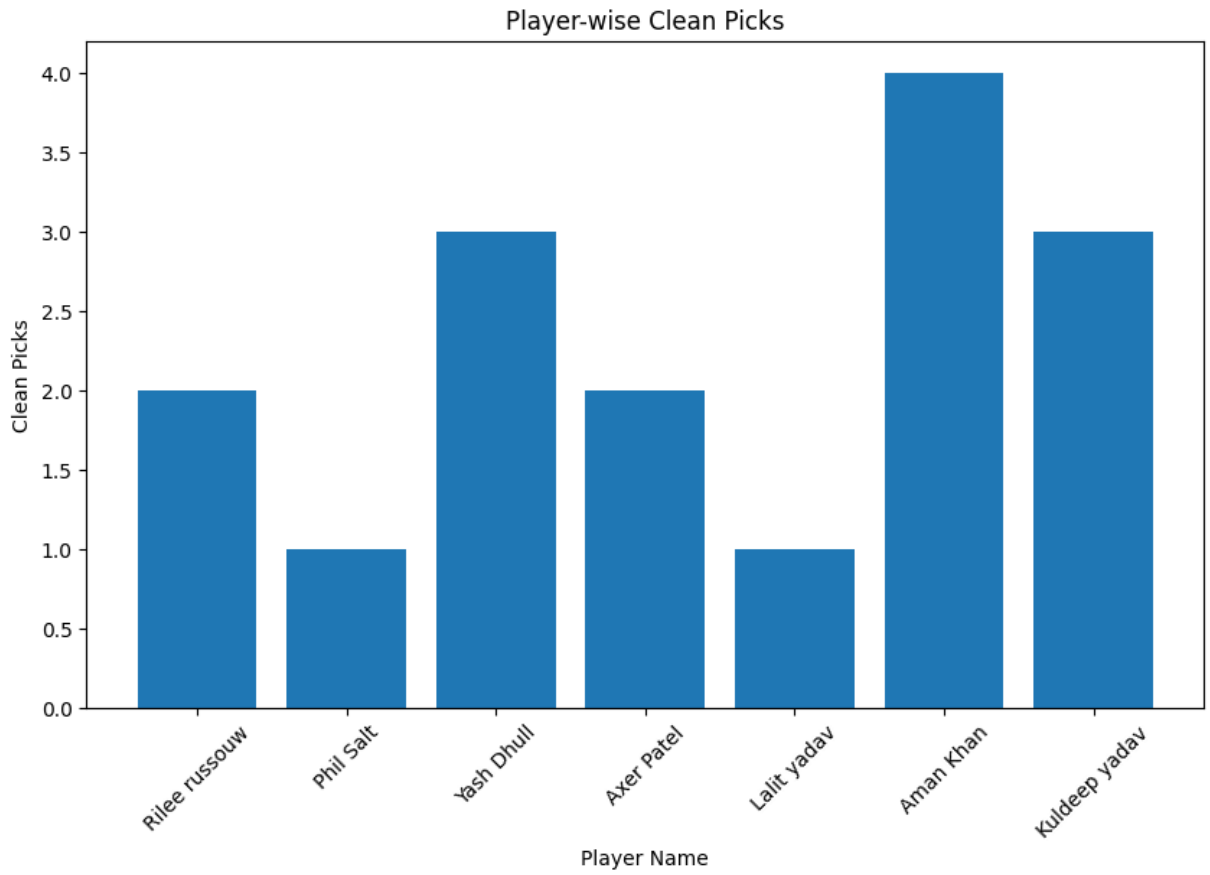
Explanation

Visualizes the selected fielding-performance measure for easier comparison.

Code

```
plt.figure(figsize=(10, 6))

plt.bar(df["Player Name"], df["CP"])

plt.title("Player-wise Clean Picks")
plt.xlabel("Player Name")
plt.ylabel("Clean Picks")

plt.xticks(rotation=45)

plt.show()
```

Output



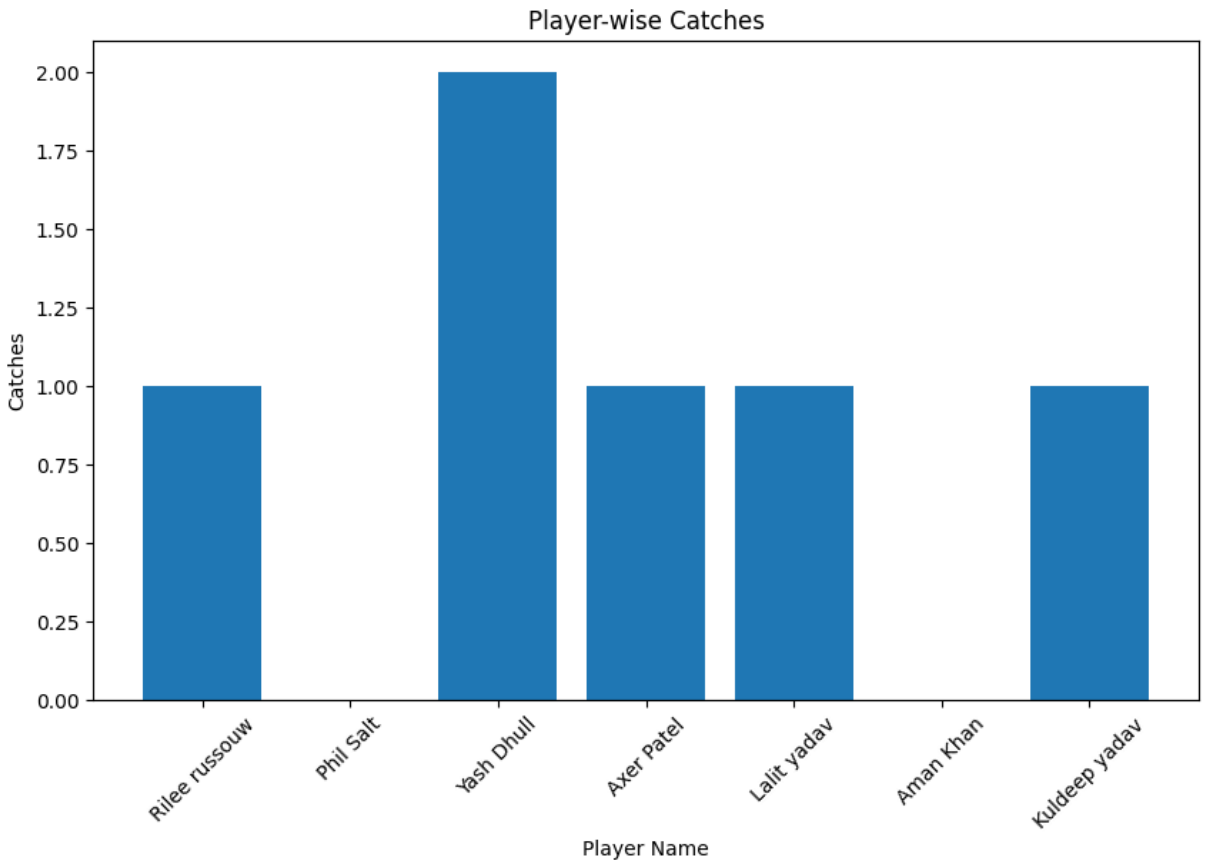
Explanation

Visualizes the selected fielding-performance measure for easier comparison.

Code

```
plt.figure(figsize=(10, 6))

plt.bar(df["Player Name"], df["C"])

plt.title("Player-wise Catches")
plt.xlabel("Player Name")
plt.ylabel("Catches")

plt.xticks(rotation=45)

plt.show()
```

Output



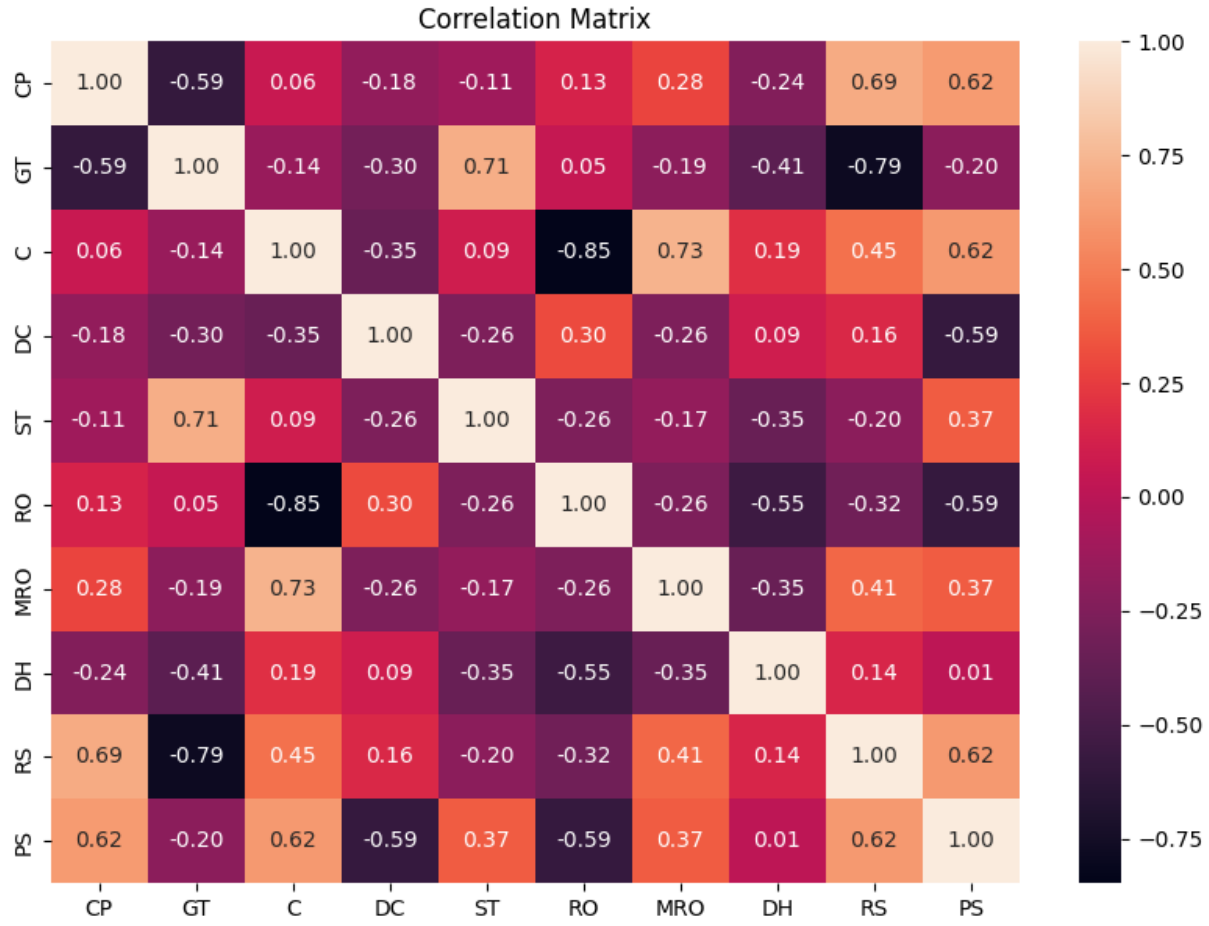
Explanation

Visualizes the selected fielding-performance measure for easier comparison.

Code

```
plt.figure(figsize=(10, 7))

corr = df[
    ["CP", "GT", "C", "DC", "ST", "RO", "MRO", "DH", "RS", "PS"]
].corr()

sns.heatmap(
    corr,
    annot=True,
    fmt=".2f"
)

plt.title("Correlation Matrix")

plt.show()
```

Output



Explanation

Visualizes the selected fielding-performance measure for easier comparison.

Top Performers

Code

```
ranking = df[
    ["Player Name", "PS"]
].sort_values(
    by="PS",
    ascending=False
)

ranking
```

Output

```
     Player Name  PS
2     Yash Dhull  11
3     Axer Patel  11
0  Rilee russouw  10
6  Kuldeep yadav   9
5      Aman Khan   9
4    Lalit yadav   6
1      Phil Salt   2
```

Explanation

Ranks or selects the top-performing players using Performance Score.

Code

```
top_3 = df.nlargest(3, "PS")

top_3[["Player Name", "PS"]]
```

Output

```
     Player Name  PS
2     Yash Dhull  11
3     Axer Patel  11
0  Rilee russouw  10
```

Explanation

Ranks or selects the top-performing players using Performance Score.

Code

```
final_table = df[
    [
        "Player Name",
        "CP",
        "GT",
        "C",
        "DC",
        "ST",
        "RO",
        "MRO",
        "DH",
        "RS",
        "PS"
    ]
].sort_values(
    by="PS",
    ascending=False
)

final_table
```

Output

	Player Name	CP	GT	C	DC	ST	RO	MRO	DH	RS	PS
2	Yash Dhull	3	1	2	0	0	0	1	0	3	11
3	Axer Patel	2	3	1	0	1	0	0	0	0	11
0	Rilee russouw	2	1	1	0	0	0	0	1	2	10
6	Kuldeep yadav	3	0	1	1	0	0	0	1	4	9
5	Aman Khan	4	1	0	0	0	1	0	0	1	9
4	Lalit yadav	1	2	1	0	0	0	0	1	-2	6
1	Phil Salt	1	2	0	1	0	1	0	0	-1	2

Explanation

Ranks or selects the top-performing players using Performance Score.

Project Summary

The complete Advance Project workflow includes library imports, dataset loading, inspection, data-quality checks, Performance Score calculation, ranking, visualizations and fielding-performance analysis.

Conclusion

The analysis provides a numerical and visual comparison of IPL fielding performance using the Performance Score and the fielding-action variables present in the dataset.